Pràctica Feed Forward: Secció OpenMP

Computació d'Altes Prestacions

Sergi Palau Travé - 1750517
Nicolás Merayo Martos - 1752648

19/10/2026

Índex de continguts

1 Introducció 3
2 Estratègia de Paral · lelització 4
2.1 init_deltas() 4
2.2 update_weights() 4
2.3 forward_prop() 5
2.4 back_prop() 6
2.5 Notes 8
3 Anàlisi de rendiment 9
4 Problemes trobats 12
5 Ús de Intel · ligència Artificial 13

1 Introducció

L'objectiu principal d'aquesta pràctica és paral · lelitzar l'apartat d'entrenament d'una xarxa neuronal de tipus *FeedForward Neural Network* programada en llenguatge C, on ens centrarem en el model de programació en memòria compartida OpenMP

Aquesta xarxa neuronal té el propòsit de classificar i reconèixer patrons en imatges de dígits escrits a mà del 0 al 9, extretes d'un dataset.

Per a compilar el programa en C, utilitzem el compilador gcc, afegint-hi la flag -fopenmp per a habilitar les regions paral · leles d'OpenMP. Ara bé, la compilació s'automatitza conjuntament dins de l'script d'enviament al gestor de tasques del servidor. Això garanteix que el codi es compili directament al node de còmput del servidor configurat amb la mateixa arquitectura expressament per a garantir una mesura correcta del temps d'execució.

Aleshores, a l'hora d'executar només cal cridar la comanda `sbatch OMP.sub <id> <fils>` o `sbatch SIMD.sub <id> <fils>` i quan el treball es finalitzi correctament es desarà automàticament la sortida del programa en un fitxer **.out**. Així mateix, hem volgut gestionar un directori auxiliar de logs per a poder desar i gestionar els diferents resultats obtinguts durant el procés de paral · lelització de les funcions.

2 Estratègia de Paral·lelització

Per a començar amb el procés de paral · lelització de la secció de l'entrenament de la xarxa neuronal (*training.c*) s'ha seguit una estratègia incremental analitzant la dependència de les dades. L'ordre de desenvolupament s'ha estructurat progressivament: des de les funcions pures i independents (*init_deltas()* i *update_weights()*), passant pel còmput de les activacions (*forward_prop()*), fins a abordar la retropropagació de l'error (*back_prop()*)

Per determinar si val la pena paral · lelitzar una funció o secció de codi, s'ha d'analitzar el seu cost computacional en relació amb el sobrecost (overhead) d'OpenMP. Quan una funció o bucle té una càrrega de treball molt petita, el temps que necessita una infraestructura de paral · lelització supera el temps que necessita un únic nucli de la CPU per a executar tot el còmput. En aquests casos, com per exemple a la funció *compute_loss()* val més la pena mantenir la seva execució 100% seqüencial. Així mateix, en certes funcions no és idoni paral · lelitzar tota la funció, poden haver bucles on no valgui la pena fer-ho d'una forma paral · lela.

2.1 init_deltas()

Aquesta funció s'encarrega de posar a zero els vectors dels acumuladors de biaix i les matrius dels acumuladors de pesos, això per a totes les capes de la xarxa. Com que no existeix cap mena de dependència de les dades, tot treballarà independentment. Ja que el bucle extern només recorre les capes de la xarxa, dins de la regió de omp designada només paral · lelitzem els for's interns.

```
void init_deltas() {
    int i, j;
    #pragma omp parallel private(i, j)
    for (i = 0; i < num_layers - 1; i++) {
        #pragma omp for schedule(static)
        for (j = 0; j < num_neurons[i]; j++)
            lay[i].dbias[j] = 0.0;

        #pragma omp for schedule(static)
        for (j = 0; j < num_neurons[i + 1] * num_neurons[i]; j++)
            lay[i].dw[j] = 0.0;
    }
}
```

2.2 update_weights()

Aquesta funció ajusta els pesos i els biaixos de la xarxa aplicant els canvis calculats, multiplicats per la velocitat d'aprenentatge. Com que l'actualització en cada pas depèn únicament del seu propi valor i del corresponent delta, no hi ha cap mena de dependència entre les dades.

Aleshores, creem una regió paral · lela, deixant completament seqüencial el bucle extern, i simplement paral · lelitzem els interns on hi ha el gruix del treball. En el càlcul de la matriu de

pesos s'aplica la clàusula *collapse(2)* sobre els bucles niats, fusionant els dos bucles perquè es reparteixi un sol espai d'iteracions entre fils.

```
void update_weights(void) {

    #pragma omp parallel
    for (int i = 0; i < num_layers - 1; i++) {

        #pragma omp for collapse(2) schedule(static)
        for (int j = 0; j < num_neurons[i + 1]; j++)
            for (int k = 0; k < num_neurons[i]; k++)  // Update
Weights
                lay[i].out_weights[j * num_neurons[i] + k] =
                    (lay[i].out_weights[j * num_neurons[i] + k]) -
                    (alpha * lay[i].dw[j * num_neurons[i] + k]);

        #pragma omp for schedule(static)
        for (int j = 0; j < num_neurons[i]; j++)  // Update Bias
            lay[i].bias[j] = lay[i].bias[j] - (alpha *
lay[i].dbias[j]);
    }
}
```

2.3 forward_prop()

La funció s'encarrega de propagar la imatge de entrada per totes les capes de la xarxa per obtenir la resposta predita.

El primer canvi és simple, vam afegir una directriu `parallel for` al bucle interior sobre els indexs j. El codi resultant és:

```
for (int i = 1; i < num_layers; i++) {
  #pragma omp parallel for schedule(static)
  for (int j = 0; j < num_neurons[i]; j++) { ... }
}
```

Aquesta era la opció més intuïtiva havent observat el comportament de la funció. Si paral·lelitzàvem el bucle sobre les i, és a dir, sobre les capes de la xarxa, obtindríem una precisió molt dolenta, això és d'esperar ja que per processar una capa es necessiten els resultats de l'anterior. Com que cada fil està editant una posició diferent de la memòria, no tenim condicions de carrera.

En el darrer canvi, movem la directriu `parallel` al bucle superior, així només es crida un cop, en comptes de per cada capa. Això fa que cridem menys cops als fils per separar el procés i ajuntar-los després. El bucle i l'executen tots els fils, es paral·lelitzen js, `omp for` crea una barrera implícita que separa dues capes del `parallel`, bàsicament establint el ordre que han de respectar els fils. També declarem variables locals dins el bucle sobre les i, així cada thread té la seva copia. Per últim, afegim la directriu `reduction` per mantenir els resultats de la suma

que es fa al bucle sobre les k en un registre del processador. Alhora evitem que s'escrigui tants cops a memòria compartida innecessàriament. Observem el codi:

```
void forward_prop() {
    #pragma omp parallel
    for (int i = 1; i < num_layers; i++) {
        const int n_previous = num_neurons[i - 1];
        const float *actv_previous = lay[i - 1].actv;
        const float *w = lay[i - 1].out_weights;

        #pragma omp for schedule(static)
        for (int j = 0; j < num_neurons[i]; j++) {
            const float *w_row = &w[j * n_previous];
            float sum = lay[i].bias[j];
            #pragma omp simd reduction(+:sum)
            for (int k = 0; k < n_previous; k++)
                sum += w_row[k] * actv_previous[k];
            lay[i].z[j] = sum;
            if (i < num_layers - 1) lay[i].actv[j] = (sum < 0) ? 0 :
sum;
            else lay[i].actv[j] = 1 / (1 + exp(-sum));
        }
    }
}
```

A més a més, podem aprofitar que estem fent sumes parcials per cridar la vectorització de la operació del bucle k amb SIMD. En altres paraules, la vectorització del producte escalar de `w_row` i `lay[i-1].actv`.

2.4 back_prop()

Aquest mètode compara la resposta obtinguda de forward prop amb la real i calcula i suma els gradients dels pesos i el biaix segons els resultats. Ha estat la més difícil de paral · lelitzar i la que més millores de rendiment ha obtingut.

Amb una observació inicial, trobem que el bucle presenta una possible condició de carrera amb l'accés a `lay[i-1].dactv[k]`, ja que cada iteració de j opera sobre tots els valors del vector amb índexs k. Si haguèssim provat afegint `atomic` o `critical`, el rendiment no hagués millorat, ja que tots els fils esperaríen per accedir al mateix vector. Per tant, creem una copia per a cada fil i afegim un `reduction`, al final del bucle, les copies de cada thread es sumen. Per al `reduction`, com `dactv` és un punter, afegim també una copia de la longitud del vector com a argument.

```
void back_prop(int p) {
  ...
  // Hidden Layers
  for (int i = num_layers - 2; i > 0; i--) {
        float *dactv_previous = lay[i - 1].dactv;
        int n_previous = num_neurons[i - 1];

        if ( i > 1 )
```

```
            for (int k = 0; k < num_neurons[i - 1]; k++)
                lay[i - 1].dactv[k] = 0;

        #pragma omp parallel for schedule(static)
reduction(+:dactv_previous[:n_previous]) // punter + longitud del
vector
        for (int j = 0; j < num_neurons[i]; j++) {
            lay[i].dz[j] = (lay[i].z[j] >= 0) ? lay[i].dactv[j] : 0;

            for (int k = 0; k < num_neurons[i - 1]; k++) {
                lay[i - 1].dw[j * num_neurons[i - 1] + k] +=
                    lay[i].dz[j] * lay[i - 1].actv[k];

                if (i > 1)
                    dactv_previous[k] +=
                        lay[i - 1].out_weights[j * num_neurons[i -
1] + k] *
                        lay[i].dz[j];
            }
            lay[i].dbias[j] += lay[i].dz[j];
        }
    }
  }
```

Més endavant observem com la iteració $i = 1$ no necessita accés a `dactv`. Per tant podem separar aquesta iteració i treure el `reduction`, evitant copiar un vector de grans dimensions a cada thread. També, com hem tret el `if`, podem vectoritzar amb SIMD el bucle sobre k. Declarem `dz` constant perquè així es llegeix i s'escriu un sol cop, en comptes de carregar de `lay[i].dz[j]` per cada k.

```
// Hidden Layers
for (int i = num_layers - 2; i > 1; i--) { // fins a capa 2 només
    float *dactv_previous = lay[i - 1].dactv;
    const int n_previous = num_neurons[i - 1];
    const float *actv_previous = lay[i - 1].actv;
    for (int k = 0; k < n_previous; k++) dactv_previous[k] = 0;

    #pragma omp parallel for schedule(static) reduction(+:
dactv_previous[:n_previous])
    for (int j = 0; j < num_neurons[i]; j++) {
        const float dz = (lay[i].z[j] >= 0) ? lay[i].dactv[j] : 0;
        float *dw_row = &lay[i - 1].dw[j * n_previous];
        const float *w_row = &lay[i - 1].out_weights[j *
n_previous];
        lay[i].dz[j] = dz;
        #pragma omp simd
        for (int k = 0; k < n_previous; k++) {
            dw_row[k] += dz * actv_previous[k];
            dactv_previous[k] += w_row[k] * dz;
        }
        lay[i].dbias[j] += dz;
```

```
    }
}
// Capa 1
{
    const int n_previous = num_neurons[0];
    const float *actv_previous = lay[0].actv;
    #pragma omp parallel for schedule(static)
    for (int j = 0; j < num_neurons[1]; j++) {
        const float dz = (lay[1].z[j] >= 0) ? lay[1].dactv[j] : 0;
        float *dw_row = &lay[0].dw[j * n_previous];
        lay[1].dz[j] = dz;
        #pragma omp simd
        for (int k = 0; k < n_previous; k++)
            dw_row[k] += dz * actv_previous[k];
        lay[1].dbias[j] += dz;
    }
}
```

2.5 Notes

En tot els bucles paral · lelitzats s'ha fet servir `schedule(static)`. Aquesta decisió ha estat ja que gairebé tots els bucles sobre les j tenen el mateix cost, totes recorren vectors de la mateixa longitud, per tant, no tenia sentit repartir la càrrega de treball d'una altra forma perquè ja estava balancejada. `static` també ens ofereix avantatges, la repartició es fa amb una sola crida al *scheduler* i ens evita mantenir una cua de tasques innecessària, cada fil rep un bloc contigu d'iteracions, així és més probable que s'aprofiti el *cache*.

3 Anàlisi de rendiment

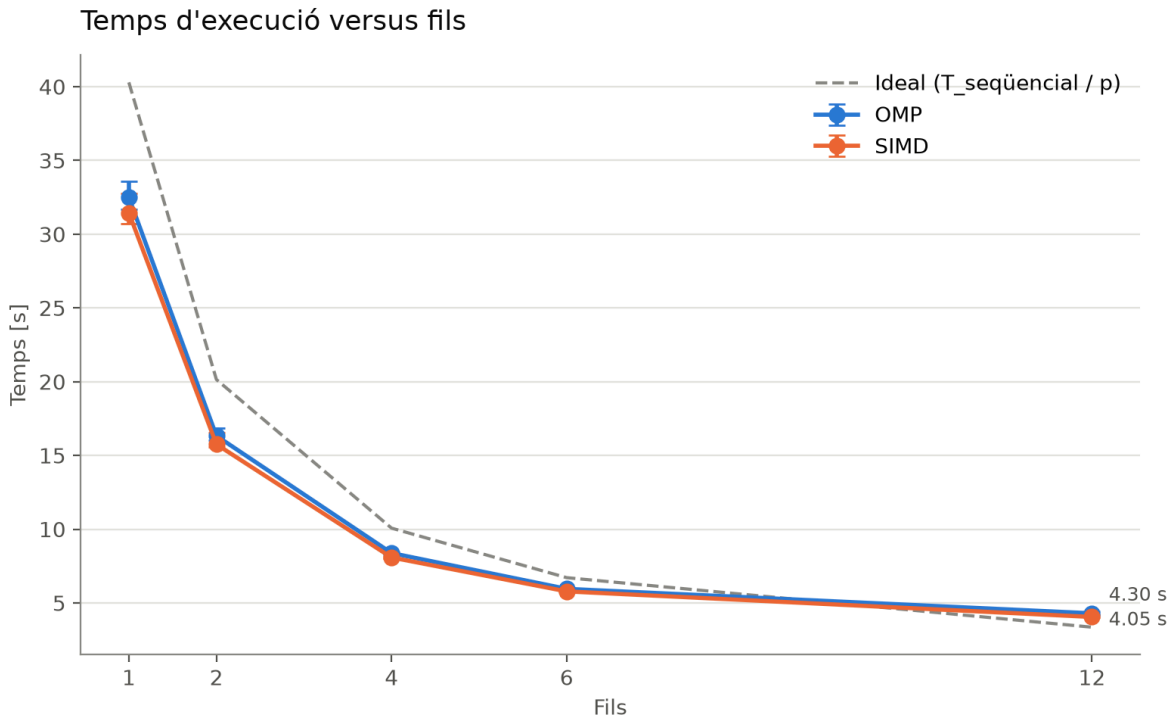
A continuació, presentem l'acceleració i l'eficiència obtingudes després d'aplicar tots els canvis. Mostrem primer els resultats del programa per a cada nombre de fils, amb i sense SIMD. Ambdues versions es compilen amb `-Ofast -fopenmp -march=native`. A la versió només amb OpenMP (OMP) afegim el flag `-fno-tree-vectorize` a GCC per evitar la vectorització, i vam comprovar amb `-fopt-info-vec-optimized` que no es vectoritza cap bucle. A la versió amb SIMD compilem amb `-DUSE_SIMD`, que activa les directives `omp simd`, i deixem la vectorització automàtica de GCC habilitada.

Per a l'acceleració prenem com a temps de referència el de la versió seqüencial original, $T_{\text{seq}} = 40.288796$ s. Per a l'eficiència, en canvi, prenem com a referència el temps del nostre codi amb un sol fil, $T_1 = 32.49$ s amb OMP i $T_1 = 31.45$ s amb SIMD, de manera que l'eficiència només mesura el que perdem en paral · lelitzar.

Versió	Fils	Execucions	μ_t	$\min t$	$\max t$	σ_t	$S(n)$	$E(n)$	$\min C$	$\max C$
OMP	1	10	32.4872	31.7042	33.6010	0.6371	1.24	1.000	928	928
OMP	2	10	16.3281	16.0438	16.8284	0.2546	2.47	0.995	928	928
OMP	4	10	8.3813	8.3442	8.4506	0.0297	4.81	0.969	927	930
OMP	6	10	5.9557	5.9299	5.9973	0.0187	6.76	0.909	928	929
OMP	12	10	4.2978	4.1532	4.4383	0.1222	9.37	0.630	927	930
SIMD	1	10	31.4473	30.7002	32.7609	0.7101	1.28	1.000	929	929
SIMD	2	10	15.7541	15.5722	16.5349	0.3103	2.56	0.998	929	929
SIMD	4	10	8.0838	8.0440	8.1753	0.0383	4.98	0.973	928	929
SIMD	6	10	5.7848	5.7627	5.8064	0.0131	6.96	0.906	927	930
SIMD	12	10	4.0506	4.0043	4.2197	0.0632	9.95	0.647	925	929

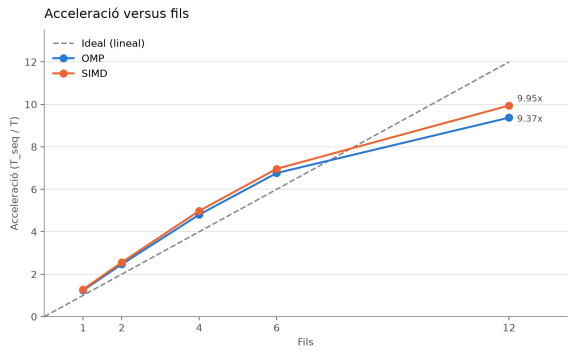
On: μ denota la mitjana, σ denota la desviació estàndard, $S(n)$ denota l'acceleració per n fils, $E(n)$ denota l'eficiència per n fils i C denota el nombre d'encerts del programa.

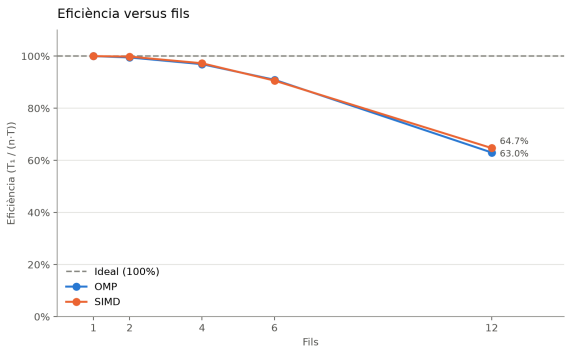
Visualitzem primer el temps contra el nombre de fils:



En aquesta figura s'observa el temps d'execució en segons respecte al nombre de fils utilitzats, en comparació amb la corba teòrica ideal. El temps d'execució disminueix ràpidament des dels 40,29 s seqüencials fins als 32,49 s amb OMP i 31,45 s amb SIMD utilitzant un sol fil, i fins als 16,33 s i 15,75 s respectivament amb 2 fils, assolint el mínim fent servir 12 fils amb 4,30 s purament amb OMP i 4,05 s amb la complementació del SIMD. La versió amb SIMD és sempre més ràpida que la d'OMP, entre un 3% i un 6% segons el nombre de fils, i la diferència més gran és la de 12 fils, on s'estalvien 0,25 s.

Observem l'acceleració i l'eficiència versus el nombre de fils:





En aquestes figures comparem per un cantó l'acceleració obtinguda, on es pot observar com en configuracions amb pocs fils (fins a 6) les corbes d'acceleració se situen per sobre de la línia ideal teòrica. Aquest comportament no s'explica per la paral · lelització, sinó perquè el nostre codi ja amb un sol fil és 1,24 vegades (OMP) i 1,28 vegades (SIMD) més ràpid que el seqüencial original, ja que s'han reestructurat els bucles de les funcions i probablement també hi influeixen

les opcions de compilació. Per això l'acceleració amb pocs fils ja surt per sobre de n i no es pot interpretar com una acceleració superlineal.

Amb 12 fils l'acceleració arriba a 9,95x amb SIMD i 9,37x amb OMP, i es situa per sota de la línia ideal teòrica, com és lògic. Aquests són els valors màxims que hem mesurat, i no podem assegurar que s'estabilitzin, ja que no hem provat amb més de 12 fils.

I per l'altre cantó parlem de l'eficiència, calculada a partir del temps del nostre codi amb un sol fil. A mesura que augmentem el nombre de fils, l'eficiència decau lleugerament fins a un 99,5% amb OMP i un 99,8% amb SIMD per a 2 fils, un 96,9% i un 97,3% per a 4 fils, i un 90,9% i un 90,6% per a 6 fils. Amb 12 fils cau fins a un 63,0% amb OMP i un 64,7% amb SIMD. Aquesta pèrdua d'eficiència s'explica pel cost de gestionar i sincronitzar els 12 fils (cada `omp for` acaba amb una barrera implícita, i el `reduction` de *back_prop()* ha de crear i sumar una còpia privada del vector per a cada fil), per les parts que es mantenen seqüencials, com la capa de sortida, i per la possible aparició de colls d'ampolla en els càlculs. No hem mesurat quina d'aquestes causes pesa més.

Pel que fa a la correcció, el nombre d'encerts varia entre 925 i 930 sobre 946, mentre que el programa seqüencial en fa 929. Amb 1 i 2 fils el resultat és sempre el mateix, i la variació apareix amb més fils. Això és d'esperar, ja que l'ordre en què el `reduction` suma les còpies privades no està especificat i les sumes de nombres reals no són associatives, de manera que cada execució acumula errors d'arrodoniment lleugerament diferents. Aquesta diferència és petita i no afecta les conclusions.

En conclusió, s'ha demostrat que la paral · lelització proposada redueix dràsticament el temps total d'entrenament (de 40,29 s a 4,05 s). Encara que fent servir 12 fils s'assoleix el temps mínim, també comporta una pèrdua important en l'eficiència del treball: amb 6 fils s'assoleix una eficiència d'un 90,6% i una acceleració de 6,96x, mentre que passar a 12 fils només redueix el temps 1,43 vegades més i fa caure l'eficiència fins a un 64,7%. Podem especular que amb un nombre de fils intermedi entre 6 i 12 trobaríem el millor compromís, però no ho hem comprovat.

4 Problemes trobats

En el procés de paral · lelització de les funcions han sorgit diversos problemes amb els quals hem hagut de lidiar. Durant el desenvolupament de la pràctica, va ser una bona decisió haver descartat la paral · lelització dels bucles externs, que només recorren les capes de la xarxa neuronal, a causa de la dependència de les dades. No és possible fer els càlculs d'una capa sense haver completat la totalitat dels de la capa anterior.

D'altra banda, es va observar que intentar paral · lelitzar la capa de sortida `//Output Layer` de la funció `back_prop()` provocava un augment del temps d'execució. Aquesta capa només disposa de 10 neurones (una per cada dígit del 0 al 9) i repartir-ne la feina entre 12 fils generava un sobrecost de sincronització i gestió molt superior a les poques operacions requerides pels càlculs. Aleshores, vam decidir mantenir aquesta capa de sortida totalment seqüencial i centrar els recursos en les capes ocultes, on hi ha el gran volum de càlculs.

Un altre aspecte rellevant va ser que, a la funció `back_prop()`, vam identificar una condició de carrera en acumular els deltes de les capes anteriors, ja que diversos fils intenten editar simultàniament les mateixes posicions de memòria. Per això, inicialment vam pensar a utilitzar les directives `atomic` o `critical`, però van resultar ser ineficients. Això va acabar generant un fort coll d'ampolla en el càlcul dels deltes que augmentava el temps d'execució. Llavors vam acabar optant per la clàusula `reduction()`, assignant una còpia privada del vector a cada fil, evitant bloquejos, i al final se sumen totes en el vector original.

5 Ús de Intel·ligència Artificial

Per tal de mantenir total transparència, estem obligats a documentar els àmbits pertinents on s'han fet servir models de llenguatge o altres eines d'intel · ligència artificial per a la realització d'aquest projecte:

- Per a la recerca d'altres formes de paral · lelització no natives de OpenMP, tot i estar habilitades per la llibreria, com ara bé les instruccions SIMD. Així com opcions per a desactivar dites opcions alternatives de paral · lelització, amb la finalitat de fer una comparativa justa de les capacitats de OpenMP pel seu compte.
- Per a generar scripts que processin els arxius de sortida de SLURM, en format .out per a generar un conjunt de dades més fàcil de treballar.
- Per a crear scripts en Python que generin gràfics complementaris amb les dades de sortida de SLURM, incloent la evolució del temps de execució i les corbes d'acceleració i eficiència.